EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 5
AIM: Develop a web application to fetch real-time weather data for a city. Use AJAX & a public weather API to retrieve information. Display temperature, humidity, and weather conditions dynamically.
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Dr. Yogesh Pingle

Aim: Develop a web application to fetch real-time weather data for a city. Use AJAX & a public weather API to retrieve information. Display temperature, humidity, and weather conditions dynamically.

Theory:
A weather web application provides real-time climate information by communicating with external servers using web technologies. In this experiment, AJAX (Asynchronous JavaScript and XML) is used to fetch live weather data from a public Weather API without refreshing the webpage.

The Weather API returns data in JSON format, which contains various weather parameters such as temperature, humidity, and weather description. JavaScript processes this data and updates the webpage dynamically using DOM manipulation.

AJAX improves user experience by allowing asynchronous data transfer between client and server. The user enters a city name, and the application retrieves and displays updated weather information instantly.

Role of AJAX

AJAX (Asynchronous JavaScript and XML) enables web applications to send and receive data from a server **asynchronously**, without refreshing the webpage. This improves performance and user experience. In this experiment, AJAX is implemented using the **Fetch API**, which allows HTTP requests such as GET to retrieve data from the Weather API. Although the name includes XML, modern AJAX applications mostly exchange data in **JSON format**.

Fetch API

The Fetch API is a modern JavaScript interface used to make HTTP requests. It returns a **Promise**, which allows handling asynchronous operations efficiently. Once the API response is received, it is converted into JSON format and processed further. The Fetch API simplifies AJAX implementation compared to older methods like XMLHttpRequest.

Weather API and API Key

A Weather API is a web service that provides access to weather data such as temperature, humidity, pressure, and weather conditions. Most Weather APIs require an **API key**, which acts as a unique identifier to authenticate users and track API usage. The API key ensures secure access and prevents misuse of the service.

JSON Data Processing

JSON (JavaScript Object Notation) is a lightweight data-exchange format that is easy to read and write. The Weather API sends weather information in JSON format. JavaScript accesses this data using object notation to extract required values like temperature, humidity, and weather description.

DOM Manipulation

The **Document Object Model (DOM)** represents the structure of a webpage as a tree of objects. JavaScript uses DOM manipulation techniques to dynamically update webpage content. In this application, the fetched weather details are inserted into HTML elements using DOM methods, allowing real-time updates without page reload.

Dynamic Web Page Behavior

Dynamic behavior is achieved when the webpage content changes based on user interaction. In this experiment, when the user clicks the button after entering a city name, the application fetches fresh weather data and updates the display area instantly. This makes the application interactive and responsive.

Error Handling

Error handling is an important aspect of web applications. If the user enters an invalid city name or if there is a network issue, the application displays an appropriate error message. This ensures better usability and reliability.

Algorithm / Procedure

1. Start the experiment.
2. Create an HTML file for user interface.
3. Add an input field to enter the city name.
4. Add a button to fetch weather details.
5. Use JavaScript to capture the city name entered by the user.
6. Send an AJAX request to the Weather API using fetch().
7. Receive weather data in JSON format.
8. Extract temperature, humidity, and weather condition from the response.
9. Display the fetched data dynamically on the webpage.
10. Stop the experiment.

Code:

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Experiment 5: Weather App</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

   <div class="container">
      <h2>Real-Time Weather</h2>

      <div class="input-group">
         <input type="text" id="cityInput" placeholder="Enter city name...">
         <button onclick="fetchWeather()">Search</button>
      </div>

      <div id="weatherDisplay" class="weather-info">
         <p>Enter a city to see the weather.</p>
      </div>
   </div>

   <script src="script.js"></script>
</body>
</html>
```

script.js:

```
function fetchWeather() {
    const city = document.getElementById("cityInput").value.trim();
    const displayArea = document.getElementById("weatherDisplay");

    if (city === "") {
        displayArea.innerHTML = "<p style='color:red;'>Please enter a city name.</p>";
        return;
    }

    // Your actual API key is placed here as a string
    const apiKey = "2fb5e266e1d779d2c5c9de39a0562d9c";

    // The API URL uses the dynamic 'city' variable and 'apiKey' variable
    const url =
`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=met
ric`;
```

```
    displayArea.innerHTML = "<p>Loading data...</p>";

    // Making the AJAX request using the Fetch API
    // Making the AJAX request using the Fetch API
    fetch(url)
        .then(response => {
            if (response.status === 401) {
                throw new Error("API Key is invalid or still activating (takes 1-2 hours).");
            }
            if (response.status === 404) {
                throw new Error("City not found. Check your spelling.");
            }
            if (!response.ok) {
                throw new Error("Something went wrong with the network.");
            }
            return response.json(); // Convert the response to JSON
        })
        .then(data => {
            // Extracting the required data
            const temp = data.main.temp;
            const humidity = data.main.humidity;
            const conditions = data.weather[0].description;

            // Dynamically updating the HTML
            displayArea.innerHTML = `
                <p><strong>City:</strong> ${data.name}</p>
                <p><strong>Temperature:</strong> ${temp} °C</p>
                <p><strong>Humidity:</strong> ${humidity}%</p>
                <p><strong>Conditions:</strong> <span style="text-transform:
capitalize;">${conditions}</span></p>
            `;
        })
        .catch(error => {
            // Handle errors (like typing a fake city name)
            displayArea.innerHTML = `<p style='color:red;'>Error: ${error.message}</p>`;
        });
}
```

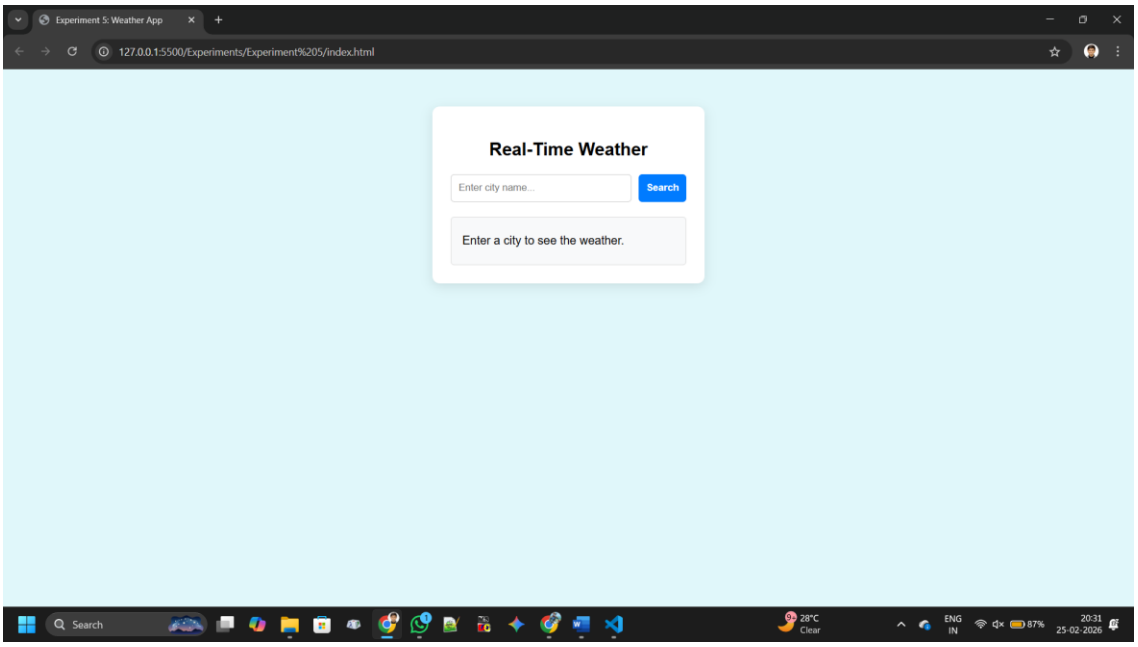
style.css:

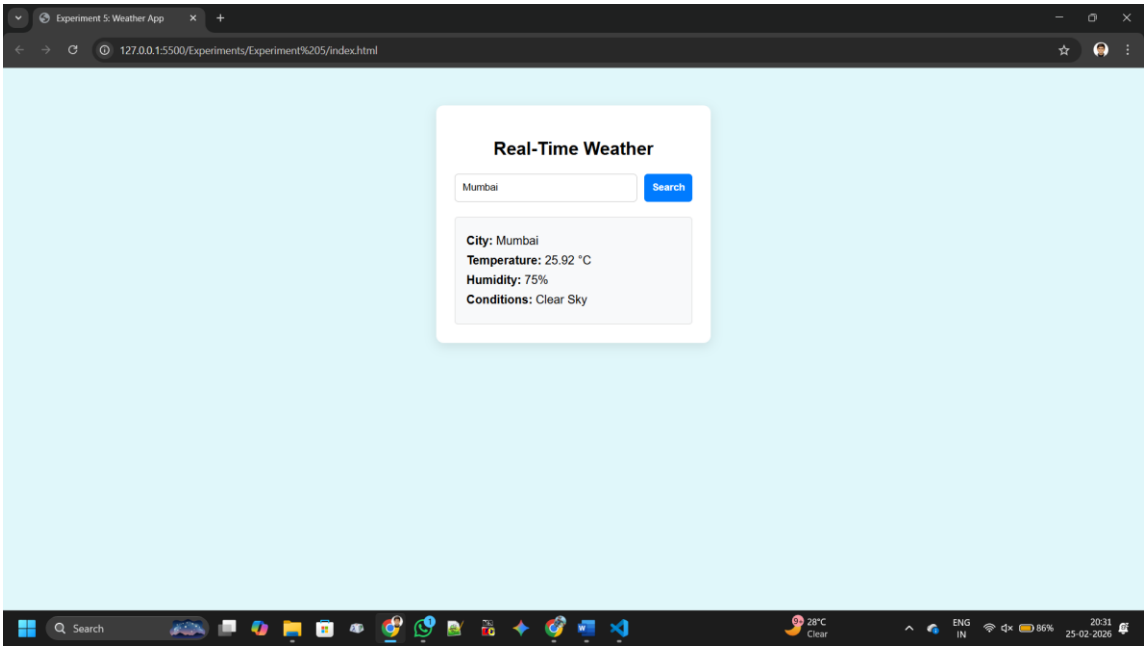
```
body {
    font-family: Arial, sans-serif;
    background-color: #e0f7fa;
    display: flex;
    justify-content: center;
```

```
    padding-top: 50px;
    margin: 0;
}

.container {
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
    width: 320px;
    text-align: center;
}

.input-group {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
}

input {
    padding: 10px;
    flex-grow: 1;
    border: 1px solid #ccc;
    border-radius: 5px;
}

button {
    padding: 10px;
    cursor: pointer;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 5px;
    font-weight: bold;
}

button:hover {
    background-color: #0056b3;
}

.weather-info {
    background: #f8f9fa;
    padding: 15px;
    border-radius: 5px;
    border: 1px solid #ddd;
    text-align: left;
}

.weather-info p {
    margin: 8px 0;
```

```
    font-size: 16px;
}
```

Output:





Conclusion

Thus, a dynamic weather web application was successfully developed using AJAX and a public Weather API. This experiment demonstrates real-time data fetching, asynchronous communication, and dynamic content updating using JavaScript and DOM manipulation

Questions:

1. **What is AJAX?**

Ans:

Definition: AJAX (Asynchronous JavaScript and XML) is a technique for sending/receiving data asynchronously without reloading the entire web page.

Key Points:

- Improves user experience with dynamic updates.
- Uses XMLHttpRequest or fetch() for communication.
- Commonly used for APIs, forms, and live data updates.

2. **What is a Weather API?**

Ans:

Definition: A Weather API provides weather-related data (temperature, humidity, forecast) from external servers.

Key Points:

- Developers can fetch real-time weather information.
- Data is usually in JSON or XML format.
- Example: OpenWeatherMap API.

3. **Why is AJAX preferred over page reload?**

Ans;

Definition: AJAX allows partial page updates instead of full reloads.

Key Points:

- Faster performance and reduced bandwidth usage.
- Provides smoother user experience.
    - Avoids losing user input/state during reload.

4. **Which method is used to fetch API data in JavaScript?**

Ans:

Definition: JavaScript provides built-in methods to fetch API data.

Key Points:

- fetch() **method**: Modern, promise-based approach.

  Javascript:

  ```
  fetch("https://api.example.com/data")
    .then(response => response.json())
    .then(data => console.log(data));
  ```

- **Alternative**: XMLHttpRequest (older method).